Solar Filament Segmentation 2026

An End-to-End Instance Segmentation Pipeline using YOLOv8l-Seg on the MAGFiLO Dataset

A Technical Project Report

Based on the `solaris.py` Implementation

Prepared by: Boobathi

Department of Computer Science and Engineering

September 18, 2026

This report explains, in full technical depth, every stage of the Solaris pipeline — from environment and dataset resolution, through COCO→YOLO label conversion, model training, inference, overlap removal, and Run-Length Encoded submission generation.

Abstract

Solar filaments are elongated, dark, thread-like structures observed in H-alpha images of the Sun's chromosphere. Automated detection and segmentation of filaments is important for space-weather monitoring, since filament eruptions are closely linked to Coronal Mass Ejections (CMEs) that can affect satellites, power grids, and communication systems on Earth.

This report documents **Solaris**, an end-to-end deep learning pipeline implemented in `solaris.py` that performs instance segmentation of solar filaments using the **YOLOv8l-Seg** architecture, trained and evaluated on the **MAGFiLO 1.0** dataset (Kaggle 2026 competition format). The pipeline is designed to be portable across Kaggle, Google Colab, and local environments, and covers five major stages:

(1) **Environment and path resolution** across multiple hosting platforms.

(2) **Dataset conversion** from COCO-style polygon/RLE annotations into normalized YOLO segmentation labels.

(3) **Model training** of YOLOv8l-Seg at high resolution (2048×2048) with domain-specific augmentation.

(4) **Inference** at high precision with strict confidence thresholding and duplicate suppression.

(5) **Post-processing** via priority-based overlap removal and conversion of binary masks into Run-Length Encoded (RLE) strings for Kaggle-style submission.

Each function in `solaris.py` is explained line-by-line, with the underlying mathematics (Dice score, IoU, mask normalization, RLE encoding) derived explicitly. The report closes with an evaluation of the pipeline's design choices, its limitations, and directions for future improvement.

Contents

Abstract 1

1 Introduction 2
- 1.1 What is a Solar Filament? . . . 2
- 1.2 Why Automate Filament Detection? . . . 3
- 1.3 Problem Statement . . . 3
- 1.4 Overview of the Solaris Pipeline . . . 3

2 Environment and Dataset Resolution 3
- 2.1 The Portability Problem . . . 3
- 2.2 The `resolve_paths()` Function . . . 3

3 The COCO Annotation Format 5
- 3.1 Why COCO? . . . 5

3.2 Two Segmentation Encodings 5

4 COCO → YOLO Label Conversion **5**
4.1 Why Conversion is Necessary 5
4.2 The `convert_coco_to_yolo_seg()` Function 5

5 Image Preprocessing and Augmentation **7**

6 YOLOv8l-Seg Architecture **7**
6.1 Why an Instance Segmentation Model? 7
6.2 Architectural Components 8
6.3 Why the "Large" (`l`) Variant 8

7 The Training Pipeline **8**
7.1 The `train_yolov8l_seg()` Function 8
7.2 Parameter-by-Parameter Explanation 9
7.3 The Loss Function 9
7.4 Checkpointing: `best.pt` vs `last.pt` 9

8 Inference Pipeline **9**
8.1 The `run_inference()` Function 9

9 Priority-Based Overlap Removal **10**
9.1 Why Overlaps Must Be Removed 10
9.2 The `remove_instance_overlaps()` Function 11

10 Binary Mask to Run-Length Encoding **12**
10.1 What is RLE? 12
10.2 The `mask_to_rle()` Function 12
10.3 Resizing Before Encoding 12

11 Submission CSV Generation **13**
11.1 Row Construction 13
11.2 Sanity Reporting 13

12 Evaluation Metrics: Dice Score and IoU **13**

13 The Main Entry Point and Orchestration **14**
13.1 Command-Line Interface 14
13.2 Weight Fallback Search 14

1. Introduction

1.1 What is a Solar Filament?

A solar filament is a mass of relatively cool, dense plasma suspended above the Sun's surface by magnetic field lines, appearing as a dark, thread-like structure when observed in H-alpha (656.3 nm) light against the brighter chromosphere. When the same structure is observed on the solar limb (edge) rather than on the disk, it appears bright and is called a *prominence*. Filaments trace magnetic polarity inversion lines and are of major interest to space-weather forecasters because their eruption is frequently associated with Coronal Mass Ejections (CMEs).

1.2 Why Automate Filament Detection?

Historically, filament identification has relied on manual annotation by solar physicists, which is slow, subjective, and difficult to scale to the volume of daily full-disk H-alpha imagery produced by observatories. An automated, pixel-accurate segmentation system enables:

- Consistent, reproducible filament catalogs across years of data.
- Near real-time flagging of filaments for space-weather alerting.
- Quantitative morphological studies (length, area, orientation) at scale.

1.3 Problem Statement

Given a full-disk H-alpha solar image, the task is to produce a set of pixel-accurate **instance segmentation masks**, one per filament, with no overlapping pixels between instances. This is framed as a Kaggle-style competition (*filament-segmentation-2026*) using the MAGFiLO 1.0 dataset, where predictions must be submitted as Run-Length Encoded masks in a CSV file.

1.4 Overview of the Solaris Pipeline

The `solaris.py` script implements the complete pipeline in five logical stages, each corresponding to a section of this report:

Stage	Function(s)	Purpose
1	`resolve_paths()`	Locate dataset and working directories
2	`convert_coco_to_yolo_seg()`	Convert COCO annotations to YOLO format
3	`train_yolov8l_seg()`	Train the segmentation model
4	`run_inference()`	Predict masks on test images
5	`mask_to_rle()`, `remove_instance_overlaps()`	Post-process and encode submission

2. Environment and Dataset Resolution

2.1 The Portability Problem

Machine learning notebooks are frequently developed on one platform (e.g. a local machine) and then executed on another (Kaggle kernels, Google Colab). Each platform mounts datasets at different filesystem paths. Rather than hard-coding a single path, Solaris implements an **auto-detection routine**.

2.2 The `resolve_paths()` Function

```
def resolve_paths():
    search_roots = [
        ".",
        "filament-segmentation-2026/MAGFiLO_1.0_Kaggle_2026",
        "/content",
        "/content/filament-segmentation-2026/MAGFiLO_1.0_Kaggle_2026",
        "/content/drive/MyDrive/kaggle_SF",
        ...
        "/kaggle/input/competitions/filament-segmentation-2026/...",
```

```
        "/kaggle/input/filament-segmentation-2026/..."
    ]
    for root in search_roots:
        if not os.path.exists(root):
            continue
        candidate_jsons = glob.glob(os.path.join(root, "**", "*Annotations*
            train*.json"), recursive=True)
        ...
```

Listing 2.1: Environment auto-detection

How it works

1. A list of **candidate root directories** is defined, covering local execution, Google Colab (`/content`), Colab with a mounted Google Drive, and two possible Kaggle mount points.
2. For each root that actually exists on disk, `glob.glob` with the `recursive=True` flag searches for:
   - A JSON file matching the pattern `*Annotations*train*.json` — the COCO-style training annotation file.
   - A folder literally named `train_images`.
   - A folder literally named `test_images`.
3. The **first match found** for each category is kept (guarded by `if candidate_jsons and not train_json`), so earlier entries in `search_roots` take priority.
4. A **working directory** is chosen based on which platform is detected: `/kaggle/working` for Kaggle, `/content/working_solaris` for Colab, or a local `working_solaris` folder otherwise.
5. The function returns a dictionary bundling all resolved paths, including a derived path for the YOLO-format dataset (`yolo_seg_dataset`) and the final submission CSV location.

Design Rationale

This pattern-matching approach means the exact same script can be dropped into a Kaggle kernel, a Colab notebook, or a local repository clone without modification — a form of *environment-agnostic* engineering that is common in competition code.

Submission Path Logic

```
"output_sub": os.path.join(working_dir, "submission.csv")
              if "/kaggle/working" in working_dir
              else "submission.csv"
```

Listing 2.2: Conditional submission path

The final submission CSV's location is decided by a single conditional expression rather than a fixed constant. On Kaggle, only files written under `/kaggle/working` are captured as kernel outputs and made available for competition submission, so when `working_dir` contains that substring, the submission path is explicitly nested inside it. On any other platform (Colab, local), the submission is simply written to `submission.csv` in the current directory, since there

is no equivalent output-capture restriction to satisfy.

3. The COCO Annotation Format

3.1 Why COCO?

The MAGFiLO dataset ships annotations in the COCO (Common Objects in Context) JSON schema, an industry-standard format for object detection and segmentation datasets. A COCO file has three top-level arrays relevant here:

Key	Description
`images`	List of image records, each with `id`, `file_name`, `width`, `height`.
`annotations`	List of instance records, each with `image_id`, `category_id`, and a `segmentation` field.
`categories`	List of class definitions (Solaris uses a single class: `filament`).

3.2 Two Segmentation Encodings

The `segmentation` field of an annotation can appear in one of two forms, and Solaris must handle both:

1. **Polygon list** — a Python `list` of flat coordinate arrays, e.g. `[[x1, y1, x2, y2, ..., xn, yn]]`, where each inner list is one closed polygon boundary.
2. **RLE dictionary** — a Python `dict` with `counts` and `size` fields, following the COCO run-length encoding convention, decoded via `pycocotools.mask`.

The Solaris conversion function branches on `isinstance(seg, list)` versus `isinstance(seg, dict)` to correctly process either encoding.

4. COCO → YOLO Label Conversion

4.1 Why Conversion is Necessary

YOLOv8-Seg does not consume COCO JSON directly. It expects, for every image, a plain-text label file in which each line represents one instance as:

$$\texttt{<class_id>}\ \texttt{x}_1\ \texttt{y}_1\ \texttt{x}_2\ \texttt{y}_2\ \ldots\ \texttt{x}_n\ \texttt{y}_n$$

where every coordinate is **normalized** to the range $[0, 1]$ relative to image width w and height h:

$$x_{\text{norm}} = \frac{x_{\text{pixel}}}{w}, \qquad y_{\text{norm}} = \frac{y_{\text{pixel}}}{h}$$

4.2 The `convert_coco_to_yolo_seg()` Function

```
np.random.seed(seed)
image_ids = sorted(list(images.keys()))
np.random.shuffle(image_ids)
n_val = int(len(image_ids) * val_ratio)
val_ids = set(image_ids[:n_val])
```

```
train_ids = set(image_ids[n_val:])
```

Listing 4.1: Train/validation split and setup

Reproducible Splitting

A fixed random seed (`seed=42` by default) ensures the train/validation split is **reproducible** across runs. Sorting `image_ids` before shuffling guarantees deterministic ordering prior to the shuffle, so the same seed always yields the same split. The first `n_val` shuffled IDs become the validation set; the remainder form the training set.

Case 1: Polygon Segmentation

```
if isinstance(seg, list):
    for poly in seg:
        if len(poly) < 6:
            continue
        coords = np.array(poly).reshape(-1, 2)
        norm_coords = coords / [w, h]
        norm_coords = np.clip(norm_coords, 0.0, 1.0)
        flat = " ".join([f"{x:.6f} {y:.6f}" for x, y in norm_coords])
        label_lines.append(f"0 {flat}")
```

Listing 4.2: Polygon normalization

A polygon needs at least 3 points (6 numbers: $x_1, y_1, x_2, y_2, x_3, y_3$) to enclose an area, hence the `len(poly) < 6` guard. The flat coordinate list is reshaped into an $(n, 2)$ array, divided element-wise by `[w, h]` to normalize, and clipped to $[0, 1]$ to guard against annotation rounding errors that place a coordinate fractionally outside the image bounds. Class index `0` is hard-coded because MAGFiLO has a single object category, `filament`.

Case 2: RLE Segmentation

```
elif isinstance(seg, dict):
    m = mask_utils.decode(seg)
    contours, _ = cv2.findContours(m.astype(np.uint8), cv2.RETR_EXTERNAL, cv2.
        CHAIN_APPROX_SIMPLE)
    for cnt in contours:
        if len(cnt) < 3:
            continue
        coords = cnt.squeeze(1)
        ...
```

Listing 4.3: RLE decoding and contour extraction

When the annotation is stored as RLE, `pycocotools.mask.decode` reconstructs the full binary mask (an $h \times w$ array of 0s and 1s). OpenCV's `findContours` then traces the **outer boundary** (`RETR_EXTERNAL`) of each connected region in that mask, compressing it into a polygon using the Douglas-Peucker-style `CHAIN_APPROX_SIMPLE` approximation. The resulting contour points are normalized exactly as in the polygon case. This unifies both annotation styles into one YOLO-compatible polygon format.

File and Directory Layout

For every image, the function:

1. Hard-links (or copies, if linking fails, e.g. across filesystems) the source image into `images/train/` or `images/val/`.

2. Writes the accumulated `label_lines` to a `.txt` file of the same stem name under `labels/train/` or `labels/val/` — YOLO's expected directory convention, where each image file has a matching label file located by filename stem.

Generating `data.yaml`

```
yaml_content = f"""path: {os.path.abspath(output_dir)}
train: images/train
val: images/val

names:
  0: filament
"""
```

Listing 4.4: YOLO dataset descriptor

This YAML file is the single configuration Ultralytics' YOLO needs to locate the dataset, the train/val image folders (relative to `path`), and the class-name mapping.

5. Image Preprocessing and Augmentation

Rather than a separate preprocessing function, Solaris delegates resizing, normalization, and augmentation to the Ultralytics `YOLO` training loop, configured through parameters passed to `model.train()`. The relevant augmentations are:

- **Mosaic augmentation** (`mosaic=0.5`): with 50% probability, four training images are stitched into one composite, exposing the model to varied object scales and contexts within a single forward pass.
- **Horizontal flip** (`fliplr=0.5`) and **vertical flip** (`flipud=0.5`): each applied with 50% probability. Because full-disk solar images have no fixed "up" orientation in the way natural photographs do, flipping is a physically valid augmentation for this domain.
- **Resizing to** `imgsz=2048`: images are resized (letterboxed) to a square 2048×2048 input, preserving the fine, thread-like structure of filaments that would be lost at YOLO's default 640-pixel resolution.

Internally, Ultralytics converts each label polygon into a rasterized binary mask target aligned to the network's output resolution, against which the predicted mask logits are compared during loss computation.

6. YOLOv8l-Seg Architecture

6.1 Why an Instance Segmentation Model?

Semantic segmentation labels every pixel with a class but does not separate touching or overlapping objects into distinct instances. Since filaments can be numerous and closely spaced on a solar disk, **instance segmentation** — which additionally distinguishes individual object instances — is required. YOLOv8-Seg extends the YOLOv8 detection architecture with a parallel mask-prediction branch.

6.2 Architectural Components

Block	Role
Backbone (CSPDarknet-based)	Extracts hierarchical features at decreasing spatial resolution and increasing channel depth.
Neck (PAN-FPN)	Fuses multi-scale feature maps bidirectionally so both fine detail and global context reach every detection head.
Detection Head	Predicts bounding boxes and class confidence, anchor-free.
Mask Prototype Head	Predicts a small set of prototype masks shared across all instances in the image.
Mask Coefficients	Per-instance weights that linearly combine the prototypes into the final instance mask.

The final instance mask is produced as:

$$M_i = \sigma\left(\sum_{k=1}^{K} c_{i,k} \cdot P_k\right)$$

where P_k are the K shared prototype masks, $c_{i,k}$ are instance-specific coefficients, and σ is the sigmoid activation that thresholds the linear combination into a $[0, 1]$ mask.

6.3 Why the "Large" (`l`) Variant

`yolov8l-seg.pt` is chosen over the smaller `n`/`s`/`m` variants because filaments are thin, low-contrast structures that benefit from the additional representational capacity of a deeper, wider backbone — at the cost of higher GPU memory and slower inference, which is why the training batch size is kept small (`batch=2`) at the high 2048-pixel resolution.

7. The Training Pipeline

7.1 The `train_yolov8l_seg()` Function

```
model = YOLO("yolov8l-seg.pt")
results = model.train(
    data=yaml_path,
    epochs=epochs,
    imgsz=imgsz,
    batch=batch,
    device=0,
    workers=4,
    optimizer="AdamW",
    lr0=0.001,
    lrf=0.01,
    mosaic=0.5,
    fliplr=0.5,
    flipud=0.5,
    project=project,
    name="yolov8l_2048",
    save=True,
    verbose=True
)
```

Listing 7.1: Model training invocation

7.2 Parameter-by-Parameter Explanation

Parameter	Meaning
`data`	Path to the `data.yaml` generated in the conversion stage.
`epochs`	Number of complete passes over the training set (default 30).
`imgsz=2048`	Input resolution — high enough to preserve thin filament structures.
`batch=2`	Small batch size, a direct consequence of the memory cost of 2048×2048 inputs on the large model variant.
`device=0`	Train on GPU index 0.
`workers=4`	Four parallel CPU data-loading processes to keep the GPU fed.
`optimizer="AdamW"`	Adam with decoupled weight decay, generally more stable than plain SGD for fine-tuning pretrained detectors.
`lr0=0.001`	Initial learning rate.
`lrf=0.01`	Final learning rate as a fraction of `lr0`, defining a decay schedule (cosine or linear, per Ultralytics defaults) from 0.001 down to 0.00001.
`mosaic, fliplr, flipud`	Augmentation probabilities, discussed in Chapter 5.
`project, name`	Output directory structure: `<project>/<name>/`.

7.3 The Loss Function

YOLOv8-Seg optimizes a composite loss combining box regression, classification, and mask losses:

$$\mathcal{L}_{\text{total}} = \lambda_{\text{box}}\mathcal{L}_{\text{box}} + \lambda_{\text{cls}}\mathcal{L}_{\text{cls}} + \lambda_{\text{dfl}}\mathcal{L}_{\text{dfl}} + \lambda_{\text{mask}}\mathcal{L}_{\text{mask}}$$

The mask component is typically a per-pixel binary cross-entropy computed only within the ground-truth bounding box region, encouraging sharp, well-localized instance masks without penalizing background pixels far from the object.

7.4 Checkpointing: `best.pt` vs `last.pt`

During training, Ultralytics saves two checkpoints per run: `last.pt` (weights after the final epoch) and `best.pt` (weights from the epoch with the best validation metric, typically mask mAP). Solaris explicitly returns and downstream-uses `best.pt`, since the final epoch is not guaranteed to be the best-generalizing one.

8. Inference Pipeline

8.1 The `run_inference()` Function

```
result = model.predict(
    img_path,
    imgsz=2048,
    conf=0.30,
    iou=0.00,
    max_det=100,
    verbose=False
)[0]
```

Listing 8.1: Prediction call

Reading the Image

The test image is first read in grayscale via `cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)` solely to recover its original `(h, w)` dimensions, which are needed later to resize predicted masks back to full resolution. The actual model input is handled internally by `model.predict()`, which applies YOLO's own preprocessing (resize, normalize, tensor conversion).

Key Inference Parameters

Parameter	Effect
`conf=0.30`	Only detections with confidence ≥ 0.30 are kept — a moderately strict threshold to suppress false positives.
`iou=0.00`	Sets the Non-Maximum Suppression (NMS) IoU threshold to zero, meaning *any* overlap between two predicted boxes causes the lower-confidence one to be suppressed — an aggressive de-duplication strategy, since Solaris performs its own, mask-level overlap removal afterward.
`max_det=100`	Upper bound on the number of instances returned per image.

Extracting Masks and Scores

```
result_masks_data = result.masks.data.cpu().numpy()
scores = result.boxes.conf.cpu().numpy() if hasattr(result.boxes, 'conf') else
    result.boxes.cls.cpu().numpy()
```

Listing 8.2: Extracting raw prediction tensors

`result.masks.data` is a tensor of shape (N, H_m, W_m) containing N predicted binary masks at the model's internal mask resolution. It is moved off the GPU (`.cpu()`) and converted to a NumPy array for downstream processing with OpenCV and `pycocotools`. The confidence score of each detection (`result.boxes.conf`) will drive the priority ordering used in overlap removal.

9. Priority-Based Overlap Removal

9.1 Why Overlaps Must Be Removed

Because `iou=0.00` does not fully prevent every possible mask-level overlap (NMS operates on boxes, not pixel masks), and because the competition requires that **no pixel belongs to more than one instance**, Solaris implements an explicit, deterministic overlap-resolution algorithm.

9.2 The `remove_instance_overlaps()` Function

```python
def remove_instance_overlaps(masks, scores):
    sorted_indices = np.argsort(scores)[::-1]
    sorted_masks = masks[sorted_indices]

    accumulated_mask = np.zeros(masks.shape[1:], dtype=np.uint8)
    non_overlapping_masks = []

    for mask in sorted_masks:
        clean_mask = mask & ~accumulated_mask
        non_overlapping_masks.append(clean_mask)
        accumulated_mask |= mask

    return np.array(non_overlapping_masks)
```

Listing 9.1: Confidence-priority overlap elimination

Step-by-Step Walkthrough

1. **Sort by confidence, descending** — `np.argsort(scores)[::-1]` gives the indices that would sort scores ascending; reversing gives descending order. The most confident detection is processed first and therefore given the strongest claim on any pixel it covers.
2. **Initialize an empty accumulator**, `accumulated_mask`, tracking every pixel already claimed by a higher-priority instance.
3. **For each mask, in priority order**: compute `clean_mask = mask & ~accumulated_mask`. This is a bitwise AND of the current mask with the logical NOT of the accumulator — keeping only pixels in the current mask that *no earlier (higher-confidence) instance already claimed*.
4. **Update the accumulator** with `accumulated_mask |= mask` (bitwise OR), adding the *original, unclipped* mask's footprint so that subsequent, lower-confidence masks are clipped against the true full extent of this instance.

Guarantee

By construction, after this loop, the union of all `clean_mask` arrays is pairwise disjoint: no pixel can appear in two different `non_overlapping_masks` entries, satisfying the competition's strict non-overlap requirement.

A Worked Numerical Example

Consider two overlapping 1D mask segments over 6 pixels, with mask A (score 0.9) covering pixels [2,3,4] and mask B (score 0.6) covering pixels [3,4,5]:

Pixel	0	1	2	3	4	5
Mask A (0.9)	0	0	1	1	1	0
Mask B (0.6)	0	0	0	1	1	1
Clean A (processed 1st)	0	0	1	1	1	0
Clean B (processed 2nd)	0	0	0	0	0	1

Mask A keeps its full footprint (nothing claimed yet); Mask B loses pixels 3 and 4 to A, retaining

only its exclusive pixel 5.

10. Binary Mask to Run-Length Encoding

10.1 What is RLE?

Run-Length Encoding compresses a binary mask by recording alternating run-lengths of background and foreground pixels along a flattened (column-major, "Fortran-order") traversal of the image, rather than storing every pixel individually. It is the standard compact representation required by COCO-style and Kaggle segmentation submissions.

10.2 The `mask_to_rle()` Function

```python
def mask_to_rle(binary_mask):
    rle = mask_utils.encode(np.asfortranarray(binary_mask.astype(np.uint8)))
    rle["counts"] = rle["counts"].decode("utf-8")
    return rle["counts"]
```

Listing 10.1: RLE encoding via pycocotools

Explanation

1. `np.asfortranarray(...)` reorders the mask's memory layout to **column-major (Fortran) order**, which is the traversal convention `pycocotools` expects — differing from NumPy's default row-major (C) order.
2. `mask_utils.encode(...)` (a thin wrapper around the COCO C API) walks the flattened array and emits a sequence of run lengths, internally serialized using a compact LEB128-style byte encoding.
3. The result's `counts` field is returned as raw `bytes`; `.decode("utf-8")` converts it into a plain string, suitable for a CSV cell.

Conceptual (Simple) RLE Example

For intuition, consider the simpler, human-readable RLE convention (alternating start-position/run-length pairs) applied to this small flattened mask:

0 0 1 1 1 0 0 1 1 0

This is encoded as: `start=3, length=3, start=8, length=2`, i.e. "3 3 8 2" — two runs of foreground pixels, at position 3 for 3 pixels and position 8 for 2 pixels. The actual `pycocotools` binary format is denser but conceptually equivalent: it records consecutive run lengths of alternating background/foreground.

10.3 Resizing Before Encoding

```python
m_res = cv2.resize(m.astype(np.uint8), (w, h), interpolation=cv2.INTER_NEAREST)
if m_res.sum() < 5:
    continue
```

Listing 10.2: Mask resize and trivial-fragment filtering

The model's raw output mask is at internal network resolution and must be resized back to the *original* image dimensions (w, h) before encoding, using **nearest-neighbor interpolation** (`INTER_NEAREST`) — critical for binary masks, since linear or cubic interpolation would introduce non-binary intermediate pixel values. Instances resizing down to fewer than 5 total foreground pixels are discarded as trivial noise fragments.

11. Submission CSV Generation

11.1 Row Construction

```
submission_rows.append({
    "filament_id": f"{file_stem}_{inst_idx}",
    "segmentation_rle": rle_str
})
inst_idx += 1
...
df_out = pd.DataFrame(submission_rows, columns=["filament_id", "
    segmentation_rle"])
df_out.to_csv(output_csv, index=False)
```

Listing 11.1: Building the submission DataFrame

Each detected, overlap-resolved, non-trivial instance becomes one row of the submission, identified by a composite key `<image_stem>_<instance_index>` (e.g. `img_0451_3` for the third filament detected in image `img_0451`), paired with its RLE string. The final CSV is written without a pandas row index column (`index=False`), matching Kaggle's expected submission schema.

11.2 Sanity Reporting

The script prints the total instance count and the number of *unique* images represented in the submission (recovered by stripping the trailing `_<index>` suffix from each `filament_id`), giving a quick sanity check that every test image was processed and that instance counts look reasonable before upload.

12. Evaluation Metrics: Dice Score and IoU

Although not computed directly inside `solaris.py` (Ultralytics computes them internally during validation), the two standard segmentation metrics underlying model selection are:

$$\text{Dice} = \frac{2\,|A \cap B|}{|A| + |B|} = \frac{2\,\text{TP}}{2\,\text{TP} + \text{FP} + \text{FN}}$$

$$\text{IoU} = \frac{|A \cap B|}{|A \cup B|} = \frac{\text{TP}}{\text{TP} + \text{FP} + \text{FN}}$$

where A is the predicted mask, B is the ground-truth mask, TP (true positive) is the pixel overlap, FP (false positive) is predicted-but-not-ground-truth pixels, and FN (false negative) is

ground-truth-but-not-predicted pixels. Dice and IoU are related by $\text{Dice} = \frac{2\,\text{IoU}}{1+\text{IoU}}$, so Dice always weights the overlap region more favorably than IoU for the same prediction.

13. The Main Entry Point and Orchestration

13.1 Command-Line Interface

```
parser.add_argument("--train", action="store_true")
parser.add_argument("--predict", action="store_true")
parser.add_argument("--model", type=str, default=None)
parser.add_argument("--epochs", type=int, default=30)
parser.add_argument("--imgsz", type=int, default=2048)
parser.add_argument("--batch", type=int, default=2)
```

Listing 13.1: Argument parsing

The script can be invoked in three modes:

- **Default (no flags)**: run conversion + training, then inference, using the freshly trained weights.
- `--train`: run conversion + training only, skip inference.
- `--predict`: skip training and go straight to inference, using either an explicitly supplied `--model` path or a fallback search over known checkpoint locations.

13.2 Weight Fallback Search

When running in predict-only mode without an explicit `--model`, the script checks, in order, the freshly trained run's expected path, a bundled pretrained filename, and a Kaggle-dataset-mounted pretrained weights path — allowing inference to work even on a fresh kernel that never ran training, as long as a pretrained checkpoint is attached as a Kaggle dataset.